# UNIVERSIDAD ALFONSO X EL SABIO

## ESCUELA BUSINESS & TECH

### GRADO EN INGENIERÍA INFORMÁTICA

# TRABAJO DE FIN DE GRADO

**Del Bit al Comportamiento: Redefiniendo la Frontera de la Seguridad Mediante el Aprendizaje Profundo**

**Rubén Nogueras González**

**Junio de 2026**

# UNIVERSIDAD ALFONSO X EL SABIO
## ESCUELA BUSINESS & TECH
### GRADO EN INGENIERÍA INFORMÁTICA

# DEL BIT AL COMPORTAMIENTO: REDEFINIENDO LA FRONTERA DE LA SEGURIDAD MEDIANTE EL APRENDIZAJE PROFUNDO

**ALUMNO:** RUBÉN NOGUERAS GONZÁLEZ
**N.P.:** 139174
**CÓDIGO TFG:**
**TUTOR ACADÉMICO:** SARA IGNACIO CERRATO
**FECHA DE PRESENTACIÓN:**
**BREVE DESCRIPCIÓN:**

AÑADIR ESTO CUANDO ACABE EL TFG!!!

**FIRMA DEL DIRECTOR DEL TRABAJO:** **FIRMA DEL ALUMNO:**

# Índice

**Índice de figuras** **3**

**1. Introducción** **5**

**2. Estado del Arte** **6**

**3. Marco Teórico** **7**
3.1. Detección de Anomalías . . . . . 7
3.2. Autoencoder . . . . . 7
3.3. Autoencoder Subcompleto . . . . . 8

**4. Diseño de la Arquitectura** **10**

**5. Implementación** **11**
5.1. Análisis Exploratorio de Datos EDA . . . . . 12
5.2. Preprocesamiento . . . . . 19
5.3. Autoencoder . . . . . 21
5.4. Modelo de Clasificación . . . . . 24

**6. Demo** **25**

**Bibliografía** **27**

# Índice de figuras


1. Estructura elemental de un Autoencoder. Obtenida a partir de Goodfellow et al. (2016) . . . 8
2. Estructura del Dataset CICIDS2017. Datos disponibles en Canadian Institute for Cybersecurity (2017) . . . 12
3. Distribución de datos benignos y maliciosos . . . 13
4. Distribución de clases: Datos benignos y maliciosos . . . 13
5. Distribución de datos maliciosos . . . 14
6. Distribución de clases: Datos maliciosos . . . 14
7. Dataframe de outliers . . . 16
8. Histograma, boxplot y Q-Q plot sobre la variable 'Flow Duration' . . . 16
9. Matriz de correlación inicial de los datos . . . 18
10. Matriz de correlación preprocesada de los datos . . . 20
11. Funciones ReLu y Sigmoide . . . 23
12. Comparación de MSE para datos benignos y maliciosos. . . . 23

CAPÍTULO

# 1

# Introducción

CAPÍTULO

# 2

# Estado del Arte

CAPÍTULO

# 3

# Marco Teórico

## 3.1. Detección de Anomalías

En esta sección vamos a explicar una parte vital de este proyecto, la cual consiste en la **detección de anomalías**. En el contexto de nuestro servidor web, una anomalía es un comportamiento extraño en el tráfico de red del servidor, sobre el cual nos vamos a centrar en aquellas peticiones de red fuera de lo común que se envían al servidor.

Más adelante vamos a saber diferenciar el tipo de ciberataque que se está intentando realizar por parte del atacante, pero en esta primera parte de la detección tenemos que diferenciar peticiones de red con un comportamiento benigno de uno maligno, y para ello vamos a utilizar un **autoencoder**.

## 3.2. Autoencoder

Un **autoencoder** es una red neuronal que intenta replicar los datos recibidos de su capa de entrada $x$ en la capa de salida $r$. No obstante, en esta copia no nos vamos a fijar al utilizar cualquier *autoencoder* en general, ya que nos interesa conocer el **error** que la red comete al intentar replicar los datos en la capa de salida $r$. De esta manera, si obtenemos un valor de error alto, los datos introducidos no se asemeja a los datos de entrenamiento de la red, y si obtenemos un valor de error bajo, lo contrario. Precisamente esto es en lo que nos vamos a basar para detectar logs malignos en el servidor, ya que si entrenamos la red con un set de datos benignos y comparamos el error producido por otras entradas benignas y malignas, el error será mucho mayor en los conjuntos de datos que el modelo no conoce, es decir, los datos malignos.

Según se especifica en Goodfellow et al. (2016), este tipo de redes se componen en dos partes: El primer componente se denomina **encoder** $f$ (transforma lo datos de entrada $x$ en $h$), el cual identifica las variables más significativas y descarta el ruido innecesario, y un **decoder** $g$, que obtiene la reconstrucción del modelo $r$ a partir de $h$. Esta es la estructura básica de la red y se puede ver reflejada en la siguiente imagen:

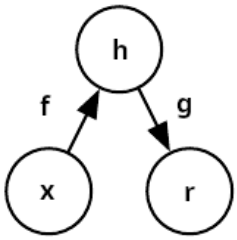


Figura 1: Estructura elemental de un Autoencoder. Obtenida a partir de Goodfellow et al. (2016)

Donde las funciones $f$ y $g$ serán definidas en la sección de implementación (3.2). Una vez se ha visto la estructura básica de un *autoencoder*, nos vamos a adentrar más específicamente en la red que se va a implementrar de manera práctica en este proyecto.

## 3.3. Autoencoder Subcompleto

Se ha visto en la sección anterior que nuestro modelo tiene que aprender de sus datos de entrenamiento, y esto se puede lograr con un **autoencoder subcompleto**. La diferencia con respecto a un autoencoder genérico es que se van a limitar los datos obtenidos en $h$, obligando a que tengan una dimensión inferior que $x$. De esta manera $h$ es forzada a aprender las propiedades más representativas de los datos de entrenamiento, lo cual resultará en una reconstrucción final $r$ con ciertas discrepancias con respecto a los datos de entrada $x$.

Podemos medir dichas discrepancias minimizando una función, la función de pérdida del modelo:

$$L(x, g(f(x))) \tag{3.1}$$

Donde $L$ es la función de pérdida que identifica si la reconstrucción del modelo $r = g(f(x))$ obtiene resultados similares a $x$. El entrenamiento de la red se basa en minimizar la función de pérdida que acabamos de ver (3.1), y la función de pérdida que se va a usar en este trabajo es el **error cuadrático medio** *MSE*, ya que penaliza los errores en mayor medida al elevar

al cuadrado, es fácil de optimizar al ser continua y derivable en todo punto del espacio (esto facilita el entrenamiento de la red), representa la distancia euclídea por lo que facilita la representación, etc.

$$MSE = \frac{1}{n} \sum_{i=0}^{n} (x_i - r_i)^2 \tag{3.2}$$

Siendo $x$ los datos de entrada, $r$ los valores de la reconstrucción, y $n$ el número total de datos en la capa de entrada y salida del modelo.

CAPÍTULO

# 4

# Diseño de la Arquitectura

CAPÍTULO

# 5

# Implementación

En esta sección vamos a aplicar lo visto en la teoría de manera práctica. Para ello se ha utilizado un set de datos del **CIC** (*Canadian Institute for Cibersecurity*), institución que nos ofrece datos de logs ya preprocesados, clasificados y con una limpieza previa para aplicarlos directamente a algoritmos de *Machine Learning* y *Deep Learning*, ahorrando bastante carga de trabajo. Los datos se encuentran disponibles en (Canadian Institute for Cybersecurity (2017)).

Para este trabajo es fundamental que los datos de entrenamiento sean de calidad, y, como bien se explica en el paper oficial Sharafaldin et al. (2018), el equipo del *CIC* se ha asegurado que estos datos sean lo más abstractos posibles. Se diseñó una infraestructura de red física que a su vez se compone de dos subredes, la red atacante y la red víctima. En la red víctima se simuló un comportamiento humano completamente normal (peticiones *HTTP/HTTPS*, correos electrónicos, *FTP*, etc.) con 25 usuarios distintos a través de diferentes máquinas y sistemas operativos para que el tráfico sea lo más realista posible.

Ante dicho tráfico legítimo, se antepusieron distintos escenarios de ataque en distintos periodos de tiempo repartidos durante los 5 días. Vamos a ver más detalladamente todos los ataques realizados durante la fase de análisis (5.1), pero entre estos destacaron *DDoS*, *DoS*, fuerza bruta, infiltración, etc. Finalmente todo el tráfico (tanto benigno como maligno) fue procesado mediante la herramienta *CICFlowMeter* para obtener los datos que se han aplicado en este proyecto, pasando tráfico de red en formato *.pcap* a *.csv*, ofreciendo una base de datos muy sólida para poder aplicar algoritmos de predicción y clasificación.

## 5.1. Análisis Exploratorio de Datos EDA

A continuación se va a comenzar con un análisis *EDA* para comprender de una mejor manera como se organizan los datos. El objetivo de esta sección es identificar las características esenciales y sacar conclusiones del conjunto de datos sin modificar, para posteriormente obtener una fuente mucho más limpia y menos redundante (si es que hubiese redundancia) en la fase de preprocesamiento (5.2). Sin más preámbulo, comenzamos con un breve vistazo al dataset:

| | Destination Port | Flow Duration | Total Fwd Packets | Total Backward Packets | ... | Idle Std | Idle Max | Idle Min | Label |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 389 | 113095465 | 48 | 24 | ... | 4277541.062 | 16500000 | 6737603 | BENIGN |
| **1** | 389 | 113473706 | 68 | 40 | ... | 4229413.12 | 16500000 | 6945512 | BENIGN |
| **2** | 0 | 119945515 | 150 | 0 | ... | 24300000.0 | 60100000 | 5702188 | BENIGN |
| **3** | 443 | 60261928 | 9 | 7 | ... | 0.0 | 0 | 0 | BENIGN |
| **4** | 53 | 269 | 2 | 2 | ... | 0.0 | 0 | 0 | BENIGN |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **2830738** | 443 | 196135 | 49 | 57 | ... | 0.0 | 0 | 0 | BENIGN |
| **2830739** | 443 | 378424 | 49 | 59 | ... | 0.0 | 0 | 0 | BENIGN |
| **2830740** | 443 | 161800 | 70 | 103 | ... | 0.0 | 0 | 0 | BENIGN |
| **2830741** | 443 | 142864 | 50 | 62 | ... | 0.0 | 0 | 0 | BENIGN |
| **2830742** | 443 | 186928 | 46 | 57 | ... | 0.0 | 0 | 0 | BENIGN |

Figura 2: Estructura del Dataset CICIDS2017. Datos disponibles en Canadian Institute for Cybersecurity (2017)

Como se puede observar, los datos constan de casi 3 millones de registros, y abarcan casi 80 columnas las cuales son todas numéricas a excepción de la última, la cual es una variable categórica que etiqueta al dataset en función de su tráfico legítimo. Es importante recalcar que los datos no se encuentran disponibles en un único dataset, sino que se encuentran separados en ficheros *.csv* según el día que se tomaron, pero para este proyecto se han juntado dichos ficheros en un único cuerpo de datos.

Entre los datos se puede diferenciar información de interés para la detección y clasificación de anomalías como información básica de conexión (*Destination Port*, *Flow Duration*), cantidad de paquetes y su tamaño (*Total Fwd Packets*, *Total Length of Fwd Packets*), velocidad (*Flow Bytes*, *Flow Packets*), Flags TCP, etc. Y la etiqueta final, que denota el flujo como benigno o maligno. Entre estas columnas se resumen la mayoría de patrones clave para detectar anomalías y, por ende, ciberataques.

Otro aspecto fundamental a revisar es la gestión de valores nulos. Aquí se vuelve a resaltar el trabajo del equipo del *CIC* por su limpieza previa, ya que no se presentan valores perdidos

en todo el flujo de datos. Asímismo, es una buena práctica separar los datos clasificados como benignos de los malignos, y esto se ve reflejado de la siguiente manera:

| | Tipo | Número de Logs | Porcentaje |
|---|---|---|---|
| 0 | Benignos | 2273097 | 80.3 |
| 1 | Malignos | 557646 | 19.7 |

Figura 3: Distribución de datos benignos y maliciosos

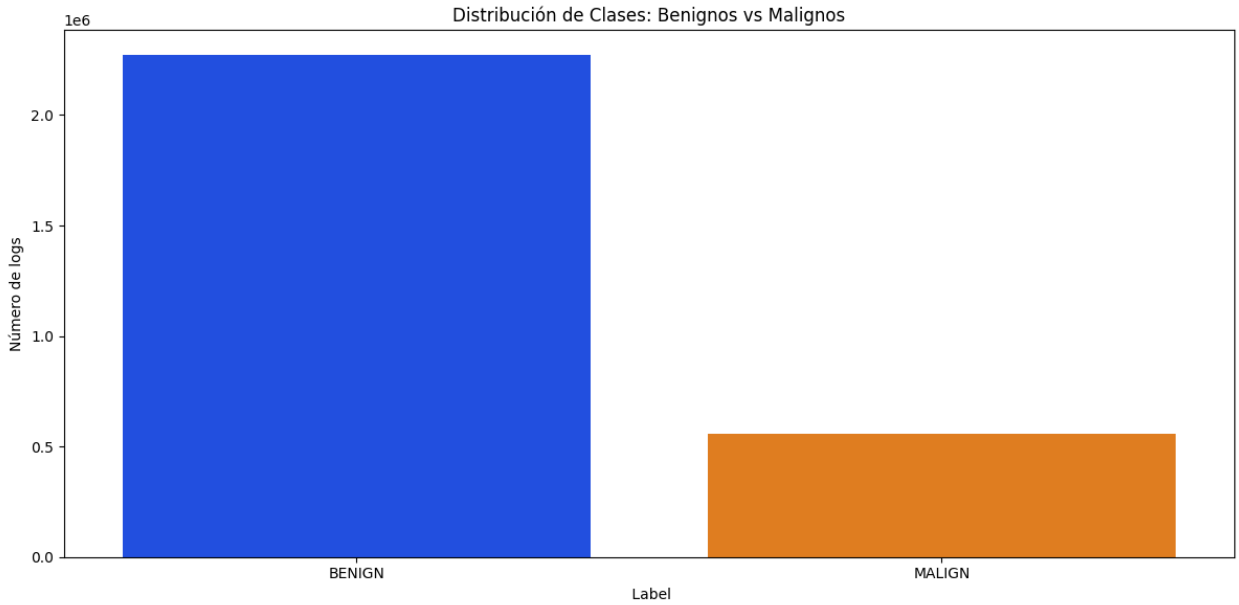


Figura 4: Distribución de clases: Datos benignos y maliciosos

Vemos claramente que aquellos datos con un fin legítimo forman un 80 % de los datos, mientras que los maliciosos un 20 %. Esto es bastante importante ya que se necesita que el autoencoder (5.3) aprenda del tráfico más usual de red para detectar anomalías con una precisión aceptable, diferenciando distintos patrones en los que se difieren los ataques. No obstante, también disponemos cerca de medio de millón de datos sobre trafico malicioso, lo que nos va a ilustrar que el autoencoder funciona correctamente detectando dicho tráfico como sospechoso, y posteriormente para entrenar un modelo de clasificación para un diagnóstico más preciso de las intenciones del atacante (5.4).

De igual manera, se ha obtenido la distribución de clase de aquellos logs maliciosos en el

dataset. Esto será de gran utilidad más adelante a la hora de evaluar el modelo de clasificación (5.4) para conocer el rango de ciberataques del que se dispone y puede clasificar dicho modelo.

| | Label | Número de Logs | Porcentaje |
|---|---|---|---|
| **0** | DoS Hulk | 231073 | 41.44 |
| **1** | PortScan | 158930 | 28.50 |
| **2** | DDoS | 128027 | 22.96 |
| **3** | DoS GoldenEye | 10293 | 1.85 |
| **4** | FTP-Patator | 7938 | 1.42 |
| **5** | SSH-Patator | 5897 | 1.06 |
| **6** | DoS slowloris | 5796 | 1.04 |
| **7** | DoS Slowhttptest | 5499 | 0.99 |
| **8** | Bot | 1966 | 0.35 |
| **9** | Web Attack \u\fffd Brute Force | 1507 | 0.27 |
| **10** | Web Attack \u\fffd XSS | 652 | 0.12 |
| **11** | Infiltration | 36 | 0.01 |
| **12** | Web Attack \u\fffd Sql Injection | 21 | 0.00 |
| **13** | Heartbleed | 11 | 0.00 |

Figura 5: Distribución de datos maliciosos

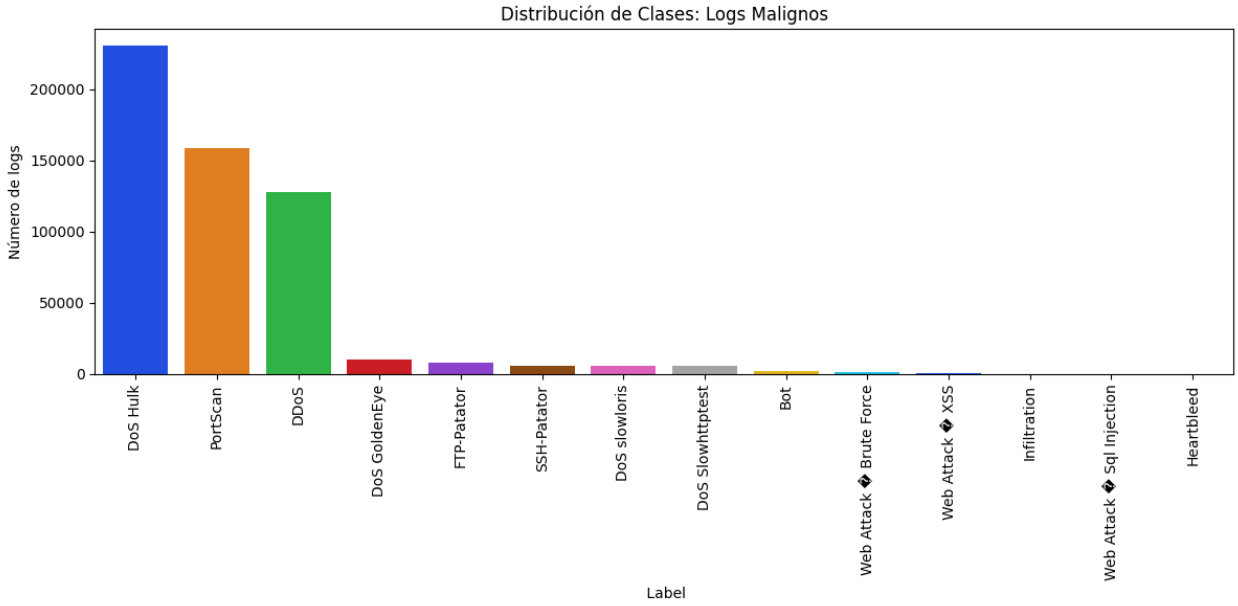


Figura 6: Distribución de clases: Datos maliciosos

Se puede apreciar que la mayoría de ataques son denegaciones de servicio, tanto denegaciones simples ($DoS$) como distribuidas por medio de varios sistemas o incluso una botnet ($DDoS$), destacando también el escaneo de puertos ($PortScan$). Este patrón indica que la mayor parte del tráfico malicioso busca saturar los recursos del sistema, mientras que otros ataques menos frecuentes tienen como objetivo una intrusión remota, filtrado de datos, obtención de credenciales, etc. Comprender esta distribución de ciberataques es un aspecto fundamental para diseñar la red y posteriormente un modelo de clasificación que funcione adecuadamente detectando la amenaza que corresponda según el contexto del servidor.

Por otra parte, no es posible llevar a cabo un *EDA* completo sin realizar un análisis de *outliers*. Un *outlier* o también conocido como valor atípico, es un dato que se aleja muy significativamente del patrón del resto de datos, por lo que afectan negativamente al rendimiento de los modelos, no solo nuestro futuro autoencoder, sino cualquier modelo de *Machine Learning* se ve afectado por esto, bien un modelo de regresión, clasificación, series temporales, etc. Es por esto que los *outliers* en ámbitos relacionados con la inteligencia artificial se suelen suprimir, aunque, como se va a ver a continuación, esto no siempre es así, y se ha de analizar de la misma manera el contexto de los datos.

Para hallar estos valores, hemos de encontrar el umbral inferior y superior de los datos. Todos los datos contenidos entre estos umbrales son datos convencionales, perfectos para entrenar modelos. No obtante, si se encuentran fuera de estos límites, son considerados atípicos y se han de gestionar más específicamente:

$$U_i = Q1 - 1{,}5\, IQR \tag{5.1}$$

$$U_s = Q3 + 1{,}5\, IQR \tag{5.2}$$

Donde $Q1$ y $Q3$ son el primer y tercer cuartil respectivamente, donde $IQR$ es el rango intercuartílico, siendo $U_i$ y $U_s$ el límite inferior y superior. Con esto, se han obtenido los siguientes *outliers*:

| | Columna | Número de Outliers | Porcentaje |
|---|---|---|---|
| **0** | Destination Port | 627245 | 22.158317 |
| **1** | Flow Duration | 532586 | 18.814354 |
| **2** | Total Fwd Packets | 332693 | 11.752851 |
| **3** | Total Backward Packets | 308291 | 10.890816 |
| **4** | Total Length of Fwd Packets | 446972 | 15.789918 |
| **...** | ... | ... | ... |
| **47** | Subflow Bwd Bytes | 619046 | 21.868675 |
| **48** | Init_Win_bytes_forward | 439029 | 15.50932 |
| **49** | Init_Win_bytes_backward | 285710 | 10.09311 |
| **50** | act_data_pkt_fwd | 310279 | 10.961045 |
| **51** | min_seg_size_forward | 2149 | 0.075916 |

Figura 7: Dataframe de outliers

Vemos que la presencia de valores atípicos es muy grande. Sin embargo no hay que preocuparse, pues esto es normal. El tráfico de una red es muy estocástico, y los datos generalmente son muy asímetricos (los datos no se ajustan siguiendo una campana de Gauss). No obstante obtenemos demasiados outliers, en parte por lo que se acaba de explicar, y porque los datos maliciosos provenientes de un ciberataque son ya por definición outliers por sí solos, aumentando aún más el número final. A continuación se presentan ciertos gráficos que apoyan estos puntos:

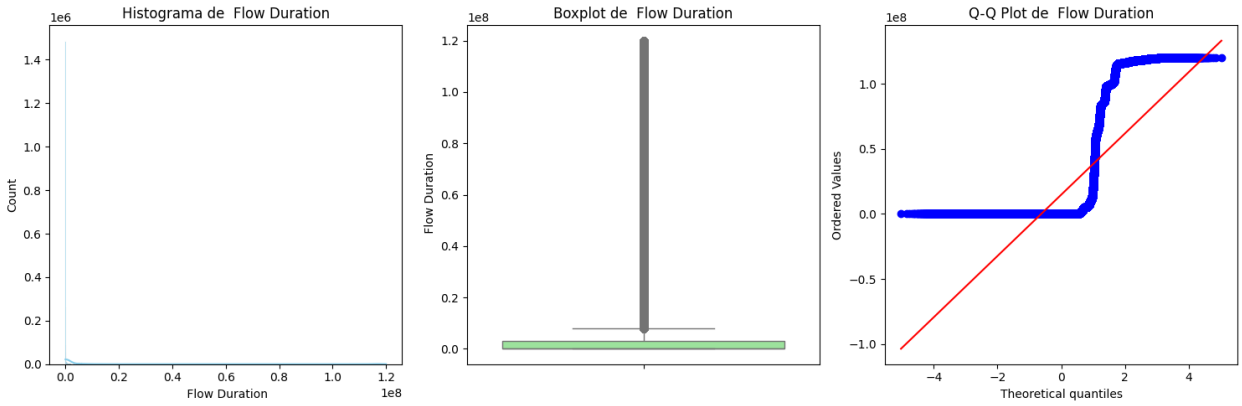


Figura 8: Histograma, boxplot y Q-Q plot sobre la variable 'Flow Duration'

En esta prueba vemos tres gráficas, un histograma, un diagrama de caja y un Q-Q plot, dichos gráficos realizados sobre la variable 'Flow Duration' del dataset, ya que esta columna es muy representativa para el modelo, al ser la duración un aspecto fundamental para explicar

el comportamiento de los datos. En el histograma se puede ver que los datos no siguen una campana de Gauss, ya que los datos se encuentran extremadamente sesgados a la derecha, donde la inmensa mayoría de registros tienen un valor muy cercano a cero, y la gran minoría restante forman una cola hasta $1{,}2 \times 10^8$.

Además, el diagrama de caja refuerza los datos que muestra el histograma, ya que la propia caja del diagrama se encuentra muy cercana a 0 (donde residen la mayoría de datos) y se muestra una gran cantidad de atípicos por encima del límite superior. Por último, el diagrama Q-Q plot termina de reforzar la teoría, viéndose claramente que los datos no siguen una distribución normal, sino que se agrupan reflejando gran cantidad de outliers.

Con esto queda justificado la presencia de multitudes valores atípicos en los datos, ya que los datos del tráfico de una red se encuentran demasiado sesgados y dispersos, negándose a seguir una distribución normal. Es por esto que se implementa un modelo autoencoder, al presentar una arquitectura no lineal que se ajusta mucho mejor a los datos, pudiendo aprender a partir de su comportamiento y características sin depender de supuestos gaussianos.

Una vez se ha visto el análisis de valores atípicos, hemos de comprobar que las distintas columnas no estén muy relacionadas entre sí, ya que el futuro modelo sería redundante y estaríamos procesando información de más que no aporta nada a la red. Para ello, obtenemos la matriz de correlación del dataset:

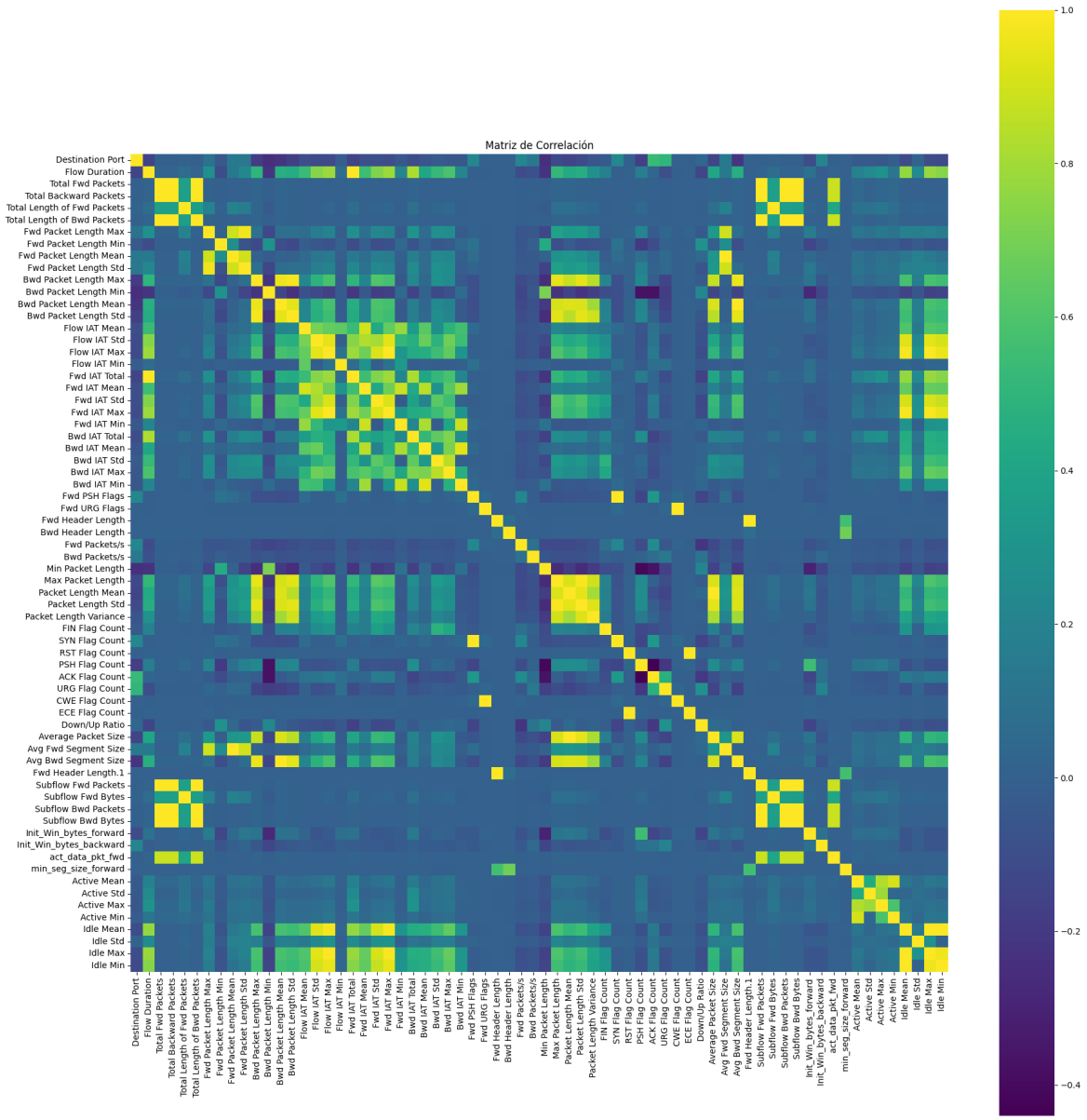


Figura 9: Matriz de correlación inicial de los datos

La matriz de correlación es muy densa, pues el dataset contiene muchas columnas de datos. Sin embargo esto no nos impide verificar la presencia de variables altamente correlacionadas entre sí. Estas distintas columnas, especialmente aquellas con una correlación muy cercana a la unidad, generan mucha redundancia al modelo, algo que no es conveniente, por lo que será uno de los primeros pasos que seguiremos en la fase de procesamiento (5.2). Con esto se da por finalizado el análisis exploratorio de datos EDA y se da paso a la sección que se acaba de mencionar.

## 5.2. Preprocesamiento

Una vez se ha comprendido de una mejor manera la naturaleza de los datos y como se distribuyen mediante un análisis exploratorio (5.1), podemos dar paso a la fase de procesamiento. El objetivo de esta sección simplemente es transformar el conjunto de datos de la manera más óptima posible para entregarlos a la capa de entrada de nuestro autoencoder. En esta fase se llevará a cabo un filtrado de variables altamente correlacionadas en los datos para reducir la redundancia de la red, y posteriormente se normalizarán los datos para mejorar el rendimiento del modelo y que se aprendan las características del tráfico benigno de una red correctamente.

Como se ha explicado, vamos a comenzar con la gestión de variables altamente relacionadas. Esto está muy ligado con el final de la sección anterior (5.1) en el que obtuvo la matriz de correlación inicial de los datos (9) pero no se llegó suprimir las variables muy correlacionadas. Para ello, se seleccionó el triángulo superior de la matriz, eliminando variables que se generaban en función de otras. Un ejemplo de esto son las columnas *'Total Length of Fwd Packets'* y *Subflow Fwd Bytes*, donde la segunda variable se obtiene a partir de la longitud total de los paquetes enviados. Así, eliminamos una de las dos variables altamente correlacionadas para reducir la redundancia, para aquella correlación superior o igual a 0,95.

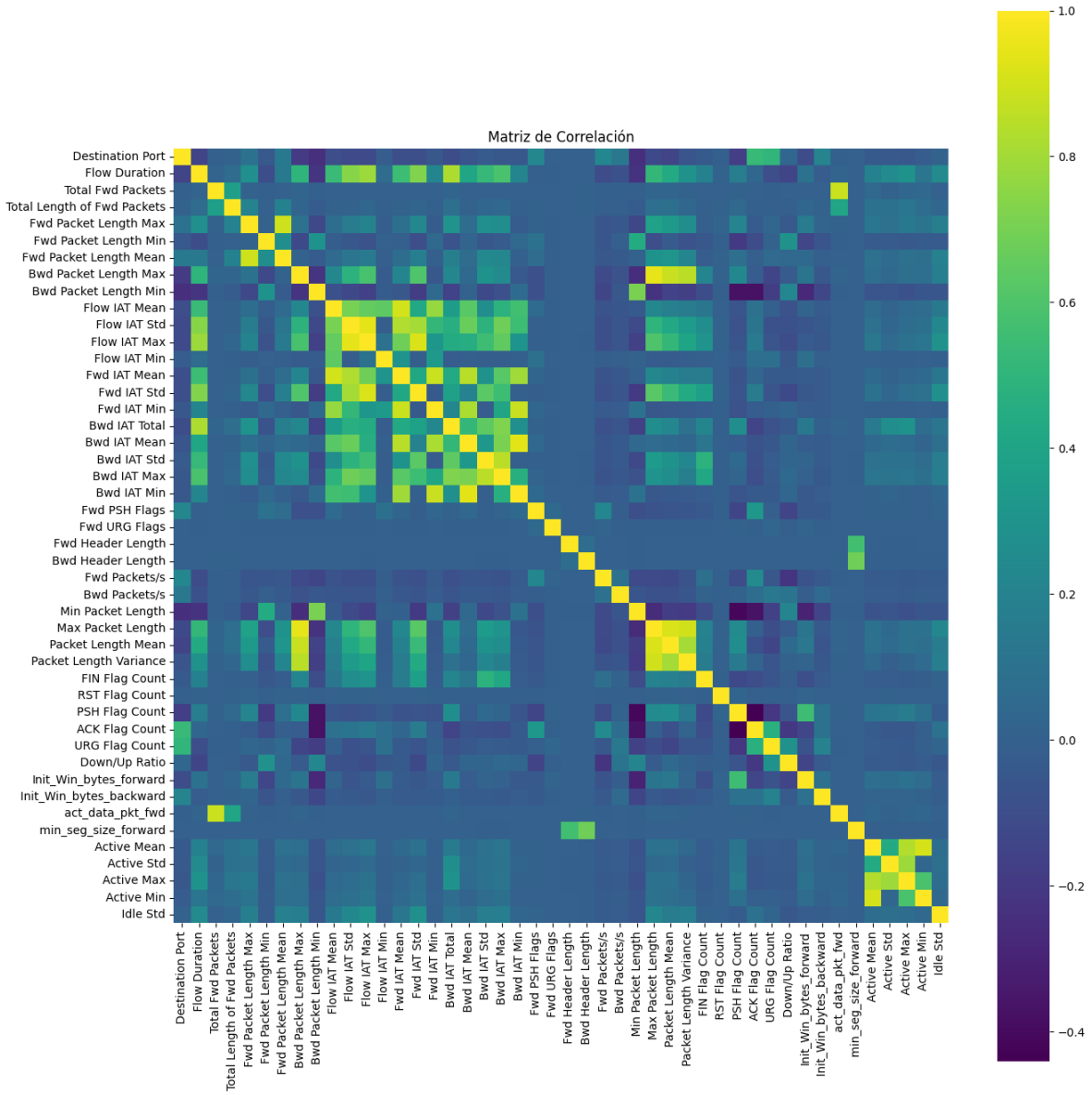


Figura 10: Matriz de correlación preprocesada de los datos

Se puede apreciar que la matriz de correlación presenta una limpieza mucho más considerable, pues se han eliminado 22 columnas repetitivas de los datos, mejorando el rendimiento de la red. Por otra parte, cabe mencionar que se encontraron valores infinitos en el dataset, los cuales por supuesto han sido correspondiente eliminados, ya que un valor infinito no es válido para una red neuronal, ahorrando futuros problemas con nuestro autoencoder.

La segunda parte de esta breve sección de preprocesamiento consiste en normalizar los datos,

consiguiendo así que todas las variables se encuentren bajo una misma escala (entre 0 y 1). Con esto se consigue que aquellas variables con valores muy altos no tengan tanta influencia durante el entrenamiento del modelo, mejorando el rendimiento y escalabilidad de nuestro autoencoder.

```
def get_scaled_train_test_data(df_to_scale):
    scaler = MinMaxScaler()
    X_train, X_test = train_test_split(
        df_to_scale,
        test_size=0.20,
        random_state=42
    )
    # Normalizamos los datos para que tengan valores entre 0 y 1
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    return X_train_scaled, X_test_scaled
```

Con esta función nos estamos ayudando de la librería Scikit-Learn de Python, disponible en [learn Developers (2026)]. Internamente, se está aplicando la siguiente transformación para el escalado de cada uno de los datos:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{max} - x_{\min}}, \qquad x_{\text{scaled}} \in [0, 1] \tag{5.3}$$

Donde $x_{\min}$ y $x_{\max}$ son los valores mínimos y máximos de cada columna. Este proceso se repite para cada columna del dataset, dejando completamente escalados los datos. Con el código anterior también dividimos los datos para el entrenamiento y test, dejando perfectamente preparados los datos para entrenar la red y verificar su correcto funcionamiento, dando paso a la siguiente sección.

## 5.3. Autoencoder

Según se vio en (3.3), el autoencoder consta de 3 capas: La capa de entrada de datos que aplica una función $f$, obteniendo las características principales para replicar los datos en una capa intermedia, para posteriormente aplicar la función $g$ y replicar los datos de entrada en la capa de reconstrucción o salida de datos. Esto se ve reflejado en el siguiente código:

```
def create_and_train_autoencoder(X_train_scaled, X_test_scaled):
    # Creamos la red neuronal (autoencoder)
    input_dim = X_train_scaled.shape[1]
    entrada = Input(shape=(input_dim,))
```

```
encoder = Dense(int(input_dim/2), activation='relu')(entrada)
decoder = Dense(input_dim, activation='sigmoid')(encoder)

autoencoder = Model(inputs=entrada, outputs=decoder)
autoencoder.compile(optimizer='adam', loss='mse')

print('Comenzamos el entrenamiento de la red...')
history = autoencoder.fit(
    X_train_scaled,
    X_train_scaled,
    epochs=20,
    batch_size=256,
    validation_data=(X_test_scaled, X_test_scaled),
    callbacks=[early_stop],
    verbose=1
)
return autoencoder, history
```

Donde vemos que las funciones de activación $f$ y $g$ son las funciones *ReLu* y *Sigmoide* respectivamente. En primer lugar se usa la función **ReLu** ya que es la función por excelencia en capas ocultas al solucionar el desvanecimiento de gradiente, y al ser lineal en los valores positivos permite a la red aprender muy rápido teniendo en cuenta las características más relevantes del tráfico de red, debido a que los valores negativos son cero.

$$f(x) = \max(0, x) \tag{5.4}$$

Por otra parte la función **Sigmoide** es perfecta para la capa de salida, ya que anteriormente se han normalizado los datos entre 0 y 1. Esto es justamente lo que hace esta función, transformar los datos a escala $[0, 1]$ debido a la presencia de asíntotas horizontales en ambos valores, lo cual es perfecto para obtener los datos de reconstrucción en esa misma escala.

$$g(x) = \frac{1}{1 + e^{-x}} \tag{5.5}$$

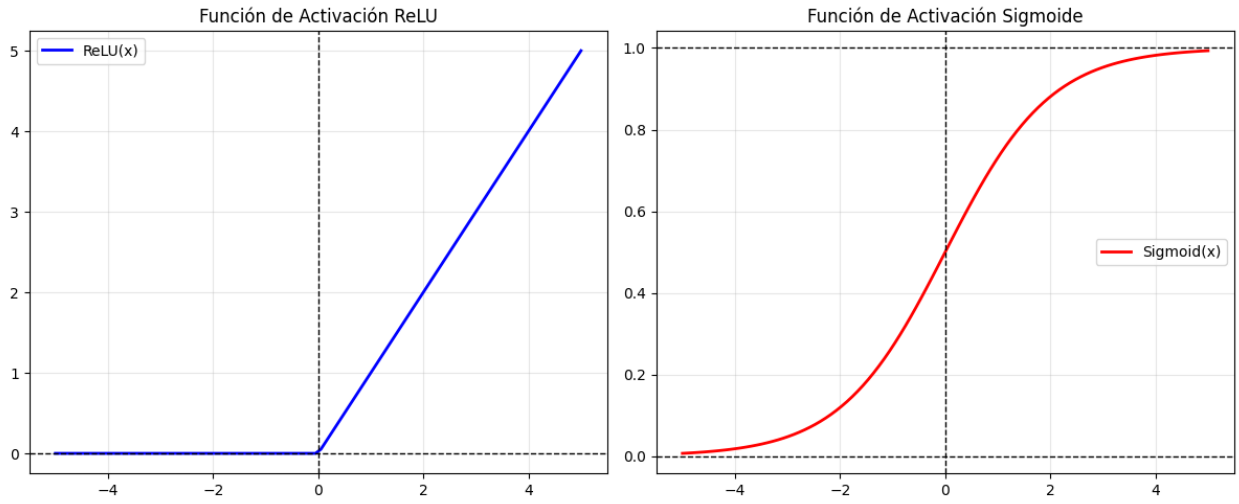


Figura 11: Funciones ReLu y Sigmoide

Una vez la fase de entrenamiento de la red ha finalizado se puede finalizar con la conclusión de los resultados. Como se vio anteriormente en (3.2), nos interesa conocer el error **MSE**, ya que en función de dicho valor se va a detectar un ataque con intenciones maliciosas. Para ello, se ha reservado un conjunto de datos clasificados como benignos (los cuales no han participado en la fase de entrenamiento de la red) y otros maliciosos, obteniendo distintas tasas de error:

| | Tipo | MSE |
|---|---|---|
| **0** | Benigno | 0.000035 |
| **1** | Maligno | 0.002665 |

Figura 12: Comparación de MSE para datos benignos y maliciosos.

Vemos que los resultados son lo que se esperaba, siendo el error casi 100 veces mayor para datos maliciosos. Esta es la funcionalidad principal del autoencoder, detectar el tráfico legítimo de paquetes de red a partir de los valores del error del modelo. Con esto concluye esta sección, una sección de vital importancia para validar la capacidad de selección de la red neuronal, verificando la correcta capacidad de aprendizaje del modelo y estableciendo la base necesaria que actúa como umbral para la detección de intrusiones y tráfico malévolo en tiempo real.

## 5.4. Modelo de Clasificación

CAPÍTULO

# 6

# Demo

# Bibliografía


Canadian Institute for Cybersecurity (2017). Cicids2017 dataset. Intrusion detection evaluation dataset including benign and attack traffic captures.

González, R. N. (2026). Repositorio del trabajo de fin de grado. https://github.com/rnoguer22/AnomaliasTFG.git. Código y recursos utilizados en el desarrollo del TFG.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA.

learn Developers, S. (2026). Scikit-learn: Machine learning in python. https://scikit-learn.org. Documentación oficial.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830.

Sharafaldin, I., Lashkari, A. H., and Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116. SCITEPRESS.